

STUDI KASUS & SOAL.

Saat promo besar pada aplikasi e-commerce terjadi:

- server overload,
- checkout gagal,
- pembayaran error.

Tim engineering melakukan:

1. monitoring,
2. scaling server,
3. rollback deployment,
4. hotfix,
5. analisis log.

Instruksi Tugas:

Lakukan analisis terhadap kasus di atas dengan menjawab pertanyaan berikut:

1. Jelaskan penyebab yang kemungkinan menyebabkan:
 - server overload,
 - checkout gagal,
 - pembayaran error.
2. Dari tindakan tim engineering:
 - monitoring,
 - scaling server,
 - rollback deployment,
 - hotfix,
 - analisis log,jelaskan fungsi masing-masing tindakan dalam proses maintenance perangkat lunak.
3. Tentukan jenis software maintenance yang dilakukan:
 - corrective,
 - adaptive,
 - perfective,
 - preventive,beserta alasannya.
4. Jika Anda menjadi software engineer pada sistem tersebut, solusi tambahan apa yang akan Anda usulkan agar masalah serupa tidak terulang?

JAWABAN.

1. Penyebab yang kemungkinan menyebabkan masalah.

a. Server overload

Server overload dapat terjadi karena lonjakan jumlah pengguna yang mengakses aplikasi secara bersamaan saat promo besar berlangsung. Tingginya jumlah permintaan (request) menyebabkan penggunaan CPU, memori, bandwidth jaringan, dan database meningkat secara drastis hingga melebihi kapasitas server yang tersedia

b. Checkout gagal

Kegagalan proses checkout dapat disebabkan oleh server yang tidak mampu memproses permintaan pengguna akibat overload. Selain itu, kemungkinan terdapat bug pada fitur checkout setelah deployment terbaru, kesalahan konfigurasi sistem, atau kegagalan komunikasi antara aplikasi dengan database sehingga data pesanan tidak dapat diproses dengan benar.

c. Pembayaran error

Pembayaran error dapat terjadi karena gangguan pada integrasi dengan payment gateway, timeout akibat tingginya trafik, kesalahan konfigurasi API pembayaran, atau bug pada modul pembayaran yang menyebabkan transaksi tidak dapat diproses atau diverifikasi dengan baik.

2. Fungsi masing-masing tindakan tim engineering dalam maintenance

a. Monitoring

Monitoring digunakan untuk memantau kondisi sistem secara real-time, seperti penggunaan CPU, memori, jumlah pengguna aktif, tingkat error, dan performa aplikasi.

b. Scaling server

Scaling server bertujuan menambah kapasitas sumber daya sistem agar mampu menangani peningkatan beban kerja. Scaling dapat dilakukan secara vertikal (menambah spesifikasi server) maupun horizontal (menambah jumlah server).

c. Rollback deployment

Rollback deployment dilakukan dengan mengembalikan sistem ke versi sebelumnya yang lebih stabil ketika versi terbaru menyebabkan bug atau gangguan. Tindakan ini membantu memulihkan layanan dengan cepat sambil menunggu perbaikan permanen.

d. Hotfix

Hotfix merupakan perbaikan cepat yang diterapkan langsung pada sistem produksi untuk mengatasi bug atau masalah kritis tanpa menunggu siklus pengembangan berikutnya.

e. Analisis log

Analisis log dilakukan dengan memeriksa catatan aktivitas sistem untuk menemukan penyebab utama masalah. Log dapat memberikan informasi

mengenai error, kegagalan transaksi, performa sistem, maupun aktivitas yang tidak normal.

3. Jenis software maintenance yang dilakukan

Tindakan	Jenis Maintenance	Alasan
Monitoring	Preventive	Dilakukan untuk mendeteksi potensi masalah sebelum menyebabkan kegagalan sistem yang lebih besar.
Scaling Server	Adaptive	Sistem disesuaikan dengan perubahan lingkungan operasional, yaitu peningkatan jumlah pengguna saat promo berlangsung.
Rollback Deployment	Corrective	Dilakukan untuk memperbaiki gangguan yang muncul akibat deployment versi baru yang bermasalah
Hotfix	Corrective	Bertujuan memperbaiki bug atau error yang telah terjadi pada sistem produksi.
Analisis Log	Preventive	Digunakan untuk mengidentifikasi akar masalah dan mencegah terulangnya gangguan yang sama di masa depan.

4. Solusi tambahan agar tidak terulang

Jika saya menjadi software engineer pada sistem tersebut, beberapa solusi tambahan yang saya usulkan adalah

- Melakukan load testing dan stress testing
Sebelum promo besar berlangsung, sistem harus diuji menggunakan simulasi ribuan hingga jutaan pengguna secara bersamaan. Pengujian ini membantu mengetahui batas kapasitas sistem dan menemukan bottleneck sebelum digunakan oleh pengguna sebenarnya.
- Menerapkan auto scalling
Menggunakan mekanisme auto scaling pada cloud sehingga jumlah server dapat bertambah atau berkurang secara otomatis sesuai tingkat trafik yang terjadi.
- Menggunakan load balancer
Load balancer dapat mendistribusikan trafik ke beberapa server sehingga tidak terjadi penumpukan beban pada satu server tertentu.
- Mengoptimalkan database
Melakukan indexing, query optimization, caching, dan database replication agar akses data tetap cepat meskipun jumlah transaksi meningkat drastis.
- Menerapkan caching
Menggunakan teknologi caching seperti Redis atau Memcached untuk mengurangi beban database dan mempercepat respons aplikasi.

- f. Menerapkan deployment yang lebih aman
Menggunakan strategi deployment seperti Blue-Green Deployment atau Canary Release sehingga perubahan dapat diuji pada sebagian kecil pengguna terlebih dahulu sebelum diterapkan secara penuh.
- g. Menyiapkan sistem monitoring dan alert otomatis
Membuat dashboard monitoring serta notifikasi otomatis ketika penggunaan sumber daya, tingkat error, atau waktu respons melebihi ambang batas tertentu.
- h. Menyediakan mekanisme failover
Menyiapkan server cadangan dan redundansi layanan pembayaran sehingga ketika satu layanan mengalami gangguan, sistem dapat secara otomatis beralih ke layanan cadangan tanpa mengganggu pengguna.